



Control of a Self-Balancing Robot (Inverted Pendulum) via State Feedback

Lucas Pin^{a,1}, Matteo Martinez^{a,1}, Franco de Vargas^{a,1}, Dr. Ing. Leonardo Comparatore^{b,1}, MSc. Ing. Christian Medina^{b,1}, MSc. Ing. Paola Maidana^{b,1}

^aMechatronics Engineering Student, Faculty of Engineering, National University of Asunción, CITEC-Luque, Paraguay

^bAdvisor, Faculty of Engineering, National University of Asunción, CITEC-Luque, Paraguay

PALABRAS CLAVE

Optimal Control
LQR
Servo System
Inverted Pendulum

RESUMEN

The modeling and control of a self-balancing robot, treated as an inverted pendulum on wheels, is developed. The process begins with the derivation of the dynamic model using the Lagrange formulation, representing the system as coupled rigid bodies. An optimal LQR state-feedback controller is then designed with the goal of stabilizing the vertical position of the system. Finally, the controller's performance is validated through simulation, demonstrating its effectiveness under different initial conditions.

1. Introduction

“Behold, wanderer of the edge, the dance of an upright body against the tyranny of gravity!”.

The control of unstable systems has been a central topic in modern control theory, with the inverted pendulum standing out as one of the most representative problems due to its highly nonlinear nature and inherent instability. In this context, the self-balancing robot offers an excellent platform for studying advanced control techniques applied to a real-world system.

The main objective of this work is to design and implement a controller capable of maintaining the balance of a two-wheeled robot with a vertical structure, using state feedback. To this end, a detailed physical model of the plant is developed, representing the various masses and mechanical components as coupled rigid bodies. From this model, the dynamic equations are derived using the Lagrange-Euler method, identifying the main state variables that influence the system's behavior.

Once the system is modeled, an optimal controller is designed using the LQR (Linear Quadratic Regulator) technique, which seeks to minimize a quadratic cost function associated with the system's performance.

The physical implementation was carried out on an Arduino platform, incorporating sensors such as a gyroscope, accelerometer, and tachometers to estimate the state variables. Control was applied using an L298N driver with a PWM signal.

Finally, the results obtained are validated both in simulation and in the physical implementation of the prototype, evaluating the effectiveness of the controller under different operating conditions.

2. Plant Description and Mathematical Modeling

2.1. Plant Description

The physical behavior of the self-balancing robot is approximated as a coupled rigid-body model, which consists of two hollow cylinders that can rotate freely about their axis (representing the wheels), and three prisms with masses m_1 , m_2 , and m_3 (Figure 1), where the fixed perpendicular distance from the center of mass of each prism to the axis about which the wheels rotate (taken as the reference $y = 0$) is given by h_1 , h_2 , and h_3 , as shown in Figure 2.

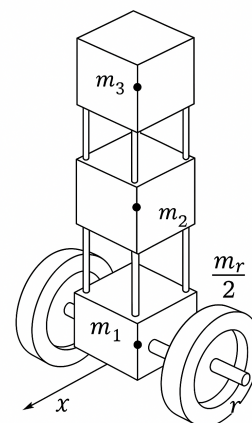


Fig. 1. Three-Dimensional Schematic of the Plant.

The assumptions made to obtain the dynamic equations are:

- The cylinders roll without slipping and both have the same linear displacement at all times.

- The axis of the cylinders and the prisms are rigidly coupled to each other.
- The mass distribution is symmetric such that the center of mass of the cylinders lies on a plane normal to the wheel axis passing through its midpoint.

These assumptions allow the system dynamics to be reduced to a two-dimensional motion.

2.2. Mathematical Modeling via the Lagrange-Euler Equations

To obtain the equations of motion, the Lagrange-Euler formulation was employed. The generalized coordinates for this approach were taken as $q_1 = x$ and $q_2 = \theta$, where x is the horizontal distance of the center of the wheel axis and θ is the pendulum angle measured from the vertical, as shown in Figure 2.

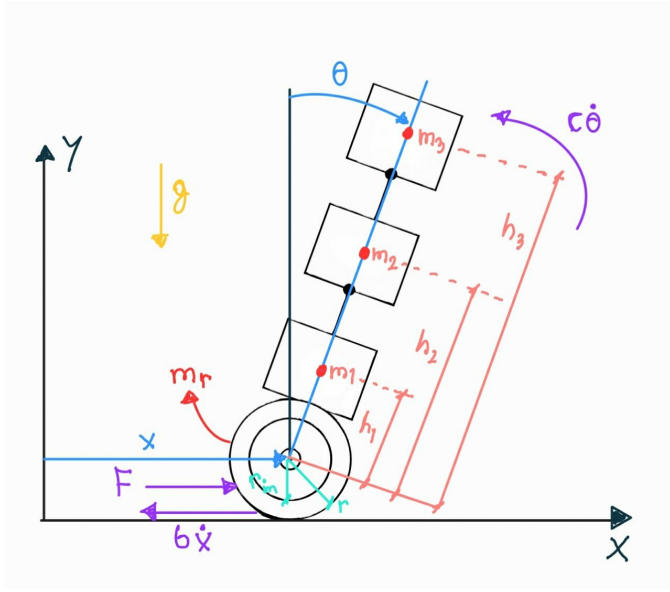


Fig. 2. Simplified Planar Diagram.

Derivation of the Kinetic Energy Expression:

⇒ Kinetic Energy of the wheels (rolling without slipping):

- Mass of each wheel: $m_r/2$.
- Translational energy: $\frac{1}{2}(m_r/2)\dot{x}^2$.
- Rotational energy (own axis): $\frac{1}{2}I_r(\dot{x}/r)^2$, with $I_r = \frac{1}{2}(m_r/2)(r + r_{int})^2$.

Therefore, the kinetic energy of the wheels is given by

$$T_{wheels} = 2 \left[\frac{1}{2} \cdot \frac{m_r}{2} \dot{x}^2 + \frac{1}{2} \cdot \frac{m_r}{2} r^2 \cdot \left(\frac{\dot{x}}{r} \right)^2 \right] = \frac{3}{4} m_r \dot{x}^2$$

⇒ Energy of the prism structure (platforms)

For each prism (platform) $i \in \{1, 2, 3\}$:

- Mass: m_i .
- Distance from CM to the wheel axis center: h_i .
- CM position: $x_i = x + h_i \sin(\theta)$, $y_i = h_i \cos(\theta)$.
- Total CM velocity:

$$v_i = \left(\frac{dx_i}{dt} \right)^2 + \left(\frac{dy_i}{dt} \right)^2 = (\dot{x} + h_i \cos(\theta) \dot{\theta})^2 + (-h_i \sin(\theta) \dot{\theta})^2$$

Expanding the expression:

$$v_i = \dot{x}^2 + 2h_i \dot{x} \dot{\theta} \cos(\theta) + h_i^2 \dot{\theta}^2$$

Therefore, the total kinetic energy of the prisms is:

$$T_{prisms} = \sum_{i=1}^3 \frac{1}{2} m_i v_i^2$$

$$T_{prisms} = \frac{1}{2} \left(\dot{x}^2 \sum_{i=1}^3 m_i + 2\dot{x} \dot{\theta} \cos(\theta) \sum_{i=1}^3 m_i h_i + \dot{\theta}^2 \sum_{i=1}^3 m_i h_i^2 \right)$$

Where:

- $m_T = \sum m_i$.
- $M_h = \sum m_i h_i$.
- $I_h = \sum m_i h_i^2$.

⇒ Total Kinetic Energy of the System:

$$T = T_{wheels} + T_{prisms}$$

$$T = \frac{1}{2} \left(m_T + \frac{3}{2} m_r \right) \dot{x}^2 + M_h \dot{x} \dot{\theta} \cos(\theta) + \frac{1}{2} I_h \dot{\theta}^2$$

Derivation of the Potential Energy Expression:

For the system considered, the potential energy consists solely of the gravitational potential energy of the masses, that is:

$$V = \sum_{i=1}^3 m_i h_i g \cdot \cos(\theta) = C_1 \cos(\theta)$$

Where g is the acceleration due to gravity and $C_1 = \sum m_i h_i g$.

Derivation of the Lagrangian:

The Lagrangian is given by:

$$\mathcal{L} = T - V$$

Therefore:

$$\mathcal{L} = \frac{1}{2} M \dot{x}^2 + M_h \dot{x} \dot{\theta} \cos(\theta) + \frac{1}{2} I_h \dot{\theta}^2 - C_1 \cos(\theta)$$

Where $M = (m_T + \frac{3}{2} m_r)$

Lagrange-Euler Equations:

⇒ For $q_1 = x$

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{x}} \right) - \frac{\partial \mathcal{L}}{\partial x} = -b\dot{x} + F$$

Where b is linear friction and F is the net force applied in x .

⇒ For $q_2 = \theta$

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\theta}} \right) - \frac{\partial \mathcal{L}}{\partial \theta} = -c\dot{\theta}$$

Where c is the angular friction.

Equations of Motion of the System:

$$\begin{aligned} (1) \quad & M \ddot{x} + M_h \cos(\theta) \ddot{\theta} = F - b\dot{x} + M_h \sin(\theta) \dot{\theta}^2 \\ (2) \quad & I_h \ddot{\theta} + M_h \cos(\theta) \ddot{x} = C_1 \sin(\theta) - c\dot{\theta} \end{aligned}$$

System Linearization:

To express the plant in state-space form, a prior linearization is required. Considering linearization around the equilibrium point $\theta = 0$, one obtains:

- $\cos(\theta) \approx 1$.
- $\sin(\theta) \approx \theta$.
- $\dot{\theta}^2 \approx 0$.

Therefore, the linearized system is:

$$\begin{aligned} (1') \quad M\ddot{x} + M_h\ddot{\theta} &= F - b\dot{x} \\ (2') \quad I_h\ddot{\theta} + M_h\ddot{x} &= C_1\theta - c\dot{\theta} \end{aligned}$$

Taking the states as:

$$X = \begin{bmatrix} x_1 & x_2 & x_3 & x_4 \end{bmatrix}^T = \begin{bmatrix} x & \dot{x} & \theta & \dot{\theta} \end{bmatrix}^T$$

It is necessary to obtain the variables $\ddot{x}$ and $\ddot{\theta}$ from (1') and (2') in terms of the state variables x_1, x_2, x_3 and x_4 . This is done by solving the matrix equation given by:

$$\begin{bmatrix} M & M_h \\ M_h & I_h \end{bmatrix} \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} F - c\dot{x} \\ C_1\theta - c\dot{\theta} \end{bmatrix}$$

Yielding the result:

$$\begin{aligned} \ddot{x} &= -\frac{I_h b}{\Delta} \dot{x} + \frac{M_h C_1}{\Delta} \theta - \frac{M_h c}{\Delta} \dot{\theta} + \frac{I_h}{\Delta} u \\ \ddot{\theta} &= \frac{M_h b}{\Delta} \dot{x} + \frac{M C_1}{\Delta} \theta - \frac{M c}{\Delta} \dot{\theta} - \frac{M_h}{\Delta} u \end{aligned}$$

Where:

- $\Delta = M I_h - M_h^2$ and $F = u$ is the control effort.

State-Space Form:

$$\dot{X} = A X + B u$$

Where matrices A and B are given by:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{I_h b}{\Delta} & \frac{M_h C_1}{\Delta} & -\frac{M_h c}{\Delta} \\ 0 & 0 & 0 & 1 \\ 0 & \frac{M_h b}{\Delta} & \frac{M C_1}{\Delta} & -\frac{M c}{\Delta} \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ \frac{I_h}{\Delta} \\ 0 \\ -\frac{M_h}{\Delta} \end{bmatrix}$$

3. Controller Design

3.1. Definition of the Continuous State Space

```

1 %% 1. PLANT MODELING
2 % Component masses in kg
3 m_bateria = 0.125; % Battery mass
4 m_arduino = 0.025; % Arduino Uno mass
5 m_driver = 0.025; % L298n driver mass
6 m_MPU = 0.002; % MPU sensor mass
7 m_tacometro = 0.04; % Mass of each tachometer + disk
8 m_rueda = 0.029; % Mass of each wheel
9 m_motor = 0.034; % Mass of each motor
10
11 g = 9.8; % Gravitational acceleration [m/s^2]
12 b = 0.1; % Linear friction
13 c = 0; % Angular friction (neglected)
14 % --- PARAMETERS (1)---
15 % Wheel data
16 mr = 2*m_rueda;
17 r = 3.4/100;
18 r_int = 2.5/100;
19 Ir = (1/2)*m_rueda*(r^2+r_int^2); %Moment of Inertia
20
21 % Prism 1 data (Battery + Motors)
22 m1 = m_bateria + 2*m_motor + 2*m_tacometro;
23 h1 = 0.1/100;
24
25 % Prism 2 data (Arduino + Drivers)
26 m2 = m_arduino + m_driver;
27 h2 = 5.2/100;
28
29 % Prism 3 data (MPU Sensor)
30 m3 = m_MPU;
31 h3 = 10/100;
32
33 % --- SIMPLIFIED PARAMETERS---
34 mT = m1 + m2 + m3;
35 Mh = m1*h1 + m2*h2 + m3*h3;
36 Ih = m1*h1^2 + m2*h2^2 + m3*h3^2;
37 M = mT + (3/2)*mr;
38 C1 = g*Mh;
39 delta = M*Ih-Mh^2;
40 %% 2. STATE-SPACE MODEL DEFINITION
41 a11 = 0;
42 a12 = 1;
43 a13 = 0;
44 a14 = 0;
45
46 a21 = 0;
47 a22 = -(Ih*b)/delta;
48 a23 = (Mh*C1)/delta;
49 a24 = -(Mh*c)/delta;
50
51 a31 = 0;
52 a32 = 0;
53 a33 = 0;
54 a34 = 1;
55
56 a41 = 0;
57 a42 = (Mh*b)/delta;
58 a43 = (M*C1)/delta;
59 a44 = -(M*c)/delta;
60
61 b1 = 0;
62 b2 = (Ih)/delta;
63 b3 = 0;
64 b4 = -(Mh)/delta;
65
66 A = [a11 a12 a13 a14;
67      a21 a22 a23 a24;
68      a31 a32 a33 a34;
69      a41 a42 a43 a44]; % System matrix A
70 B = [b1 b2 b3 b4]'; % System matrix B
71 C = [1 0 0 0]; % taking x as output
72 D = 0;

```

Yielding the following matrices:

$$A = \begin{bmatrix} 0 & 1,0000 & 0 & 0 \\ 0 & -0,2847 & 1,6946 & 0 \\ 0 & 0 & 0 & 1,0000 \\ 0 & 5,6270 & 227,1964 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ 2,8469 \\ 0 \\ -56,2702 \end{bmatrix}$$

And by taking the state $x_1 = x$ (this is the only combination that makes the system observable), the linear distance, the matrix C is given by:

$$C = [1 \ 0 \ 0 \ 0] \quad \text{and} \quad D = 0$$

3.2. Discretization Method

For the discretization, a sampling time of $T_s = 10 \text{ ms}$ was chosen, appropriate for implementation on development boards such as Arduino. The discretization method used was **ZOH (Zero-Order Hold)**, since it faithfully reflects how control signals are held constant between updates in real digital systems.

```
1 %% 2. DISCRETIZATION
2 Ts = 0.01; % Sampling time 10ms
3
4 sys_C = ss(A,B,C,D); % Continuous plant
5 sys_D = c2d(sys_C, Ts, 'zoh'); % Discrete plant
6
7 % Discrete State-Space Matrices
8 G = sys_D.A;
9 H = sys_D.B;
10 C = sys_D.C;
11 D = sys_D.D;
```

Yielding the following discrete state-space matrices:

$$G = \begin{bmatrix} 1,0000 & 0,0100 & 0,0001 & 0,0000 \\ 0 & 0,9972 & 0,0170 & 0,0001 \\ 0 & 0,0003 & 1,0114 & 0,0100 \\ 0 & 0,0564 & 2,2811 & 1,0114 \end{bmatrix}$$

$$H = \begin{bmatrix} 0,0001 \\ 0,0284 \\ -0,0028 \\ -0,5640 \end{bmatrix}$$

Having the discrete system, it can be verified that it **satisfies full controllability** and is observable only when the linear position x is taken as the feedback input (which is the case studied in this work).

3.3. Controller Design and Computation

The given system allows us to design an observable servo system. Since the feedback must be performed using the linear position x to observe the remaining variables, and this cannot be measured directly (numerical integration of the tachometer readings is required), and moreover the quality of the observer estimation depends on the fidelity of the physical model used, **it was decided to use a servo system without an observer**. The computation of the state feedback matrix K was carried out using the **LQR (Linear Quadratic Regulator)** method.

```
1 %% 4. STATE FEEDBACK DESIGN
2 % High penalty for x and theta
3 Q = diag([200, 10, 200, 20]);
4 R = 1; % Penalty for control effort
5
6 K = dlqr(G,H,Q,R);
```

Yielding the result:

$$K = [-4,7152 \quad -3,3278 \quad -17,0580 \quad -1,8229]$$

3.4. Difference Equation Defining the Closed-Loop System

The state dynamics are updated according to the following expression:

$$X[k+1] = GX[k] + H(-KX[k])$$

The control effort is:

$$u[k] = -KX[k]$$

3.5. Simulation Results

The following shows the system response without state feedback and with state feedback, for an initial inclination of $\theta = 10^\circ \approx 0,1745 \text{ rad}$. The initial state vector is thus:

$$X[0] = \begin{bmatrix} x_1[0] \\ x_2[0] \\ x_3[0] \\ x_4[0] \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0,1745 \\ 0 \end{bmatrix}$$

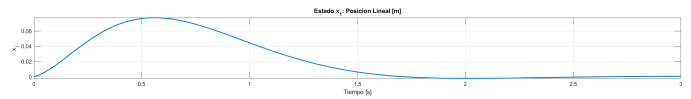


Fig. 3: Evolution of linear position [m] at an initial inclination of 10° with state feedback

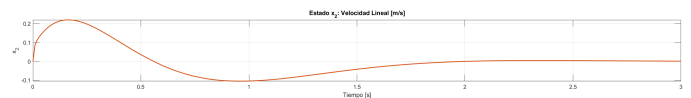


Fig. 4: Evolution of linear velocity [m/s] at an initial inclination of 10° with state feedback

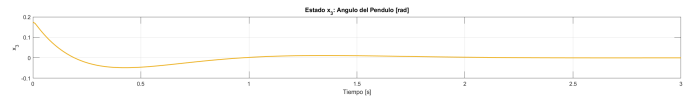


Fig. 5: Evolution of pendulum angle [rad] at an initial inclination of 10° with state feedback

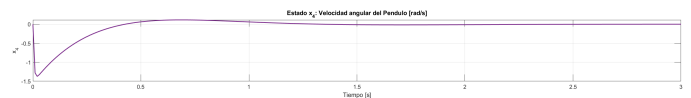


Fig. 6: Evolution of pendulum angular velocity [rad/s] at an initial inclination of 10° with state feedback

4. Controller Implementation on the Self-Balancing Robot

4.1. Sensors and Drivers Used

Three sensors were used to estimate the state variables of the system: a gyroscope, an accelerometer, and two tachometers (one per wheel). The linear velocity of the system was obtained from the average of the angular velocities measured by the tachometers, multiplied by the wheel radius. The linear position was estimated through numerical integration of this velocity.

The pendulum angle was obtained primarily from the gyroscope. However, to reduce measurement error and noise, a 1D Kalman filter was applied combining the accelerometer data, followed by a complementary filter to smooth the signal. The pendulum angular velocity was estimated from these signals via integration and additional filtering.

Motor control was implemented using an L298N driver, where the PWM input corresponds to the control effort u generated by the controller, previously scaled. This allows controlling both the direction and the magnitude of the system's movement.

4.2. Controller Implementation on Arduino

```

1 void loop() {
2   /// STATE FEEDBACK GAIN DEFINITION ///
3   float k1 = -4.7152;
4   float k2 = -3.3278;
5   float k3 = -17.0580;
6   float k4 = -1.8229;
7
8   /// ACQUISITION OF STATE VARIABLES ///
9   gyro.update(); // Updates the gyroscope reading
10  // Obtaining the angle theta
11  float Pitch_angle = gyro.get_PitchAngle();
12
13  // Obtaining the pendulum angular velocity:
14  float Pitch_rate = gyro.get_PitchAngle_Rate();
15  // Complementary filter for the angle
16  Pitch_rate = (0.6* Pitch_rate+ (1-0.6)*prev_Pitch)-3;
17  prev_Pitch= Pitch_rate;
18
19  tacoA.update(); // Updates tachometer of wheel A
20  tacoB.update(); // Updates tachometer of wheel B
21  // Takes velocity as the average
22  float w_ruedas = (tacoA.get_angular_rate()+
23                   tacoB.get_angular_rate())/2;
24  // Linear velocity of the wheels
25  float v_lineal = w_ruedas*(3.4/100);
26
27  integrador.setVelocity(sing*v_lineal);
28  integrador.update();
29  // Position is obtained by integrating the velocity
30  float x = integrador.getPosition();
31
32  /// CONTROL LAW ///
33  // 10 ms sampling
34  if (millis() - t_ant >= 10){
35
36    t_ant = millis();
37
38    // Computation of the control effort
39    u = -(x*k1+v_lineal*k2+Pitch_angle*k3+Pitch_rate*k4);
40
41  }
42
43  // Motor control with u
44  Control_Motors(u);
45 }

```

4.3. Implementation Results

```

//////////
ESTADOS
x:-0.23
v_lineal:0.60
tita:-11.80
tita_rate:-24.79
Esfuerzo de Control
u:-247.58
//////////
ESTADOS
x:-0.54
v_lineal:0.60
tita:-16.48
tita_rate:-9.10
Esfuerzo de Control
u:-299.74
//////////
ESTADOS
x:-0.80
v_lineal:0.80
tita:-1.74
tita_rate:19.39
Esfuerzo de Control
u:7.23
//////////
ESTADOS
x:-1.14
v_lineal:0.80
tita:-0.85
tita_rate:7.25
Esfuerzo de Control
u:-4.04

```

Fig. 7. Implementation Results on Arduino (θ measured in degrees)

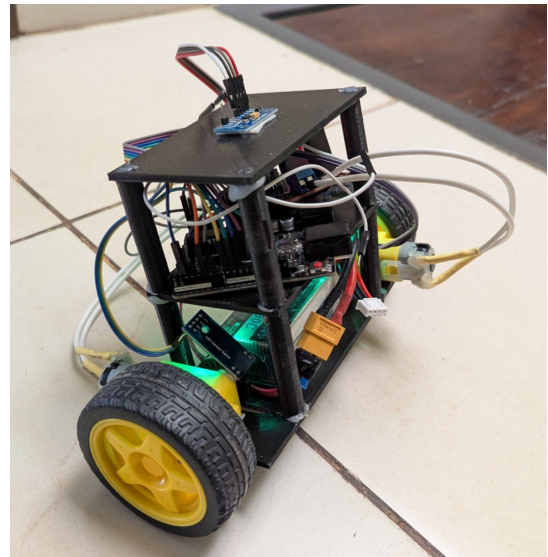


Fig. 8. Self-Balancing Robot Prototype "Paco"

5. Conclusions

This work demonstrated the effectiveness of modern control methods, such as state feedback, and highlights their great importance in MISO-type systems that are also inherently unstable, as is the case of the Self-Balancing Robot. These same systems exhibit poor or no performance under classical control techniques such as PID, which is a control strategy designed exclusively for SISO-type systems.

6. References

- [1] Katsuhiko Ogata. *Discrete-Time Control Systems*. 2nd ed., Prentice Hall, Upper Saddle River, New Jersey, 1995.
- [2] Ramírez-González, L., García-Martínez, J., Siordia-Vásquez, X. y García-Ramos, R. (2017). *Euler-Lagrange en el modelado y control de un péndulo invertido sobre un carro*. Revista de Ingeniería Eléctrica, 1(3), 1-8. ECFORFAN.